Формат растрового шрифта FMK2

Версия документа: 21.09.2026

Файл `.fmk` содержит компактный монохромный растровый шрифт для MK61s mini. Текущая версия формата имеет сигнатуру FMK2. Глифы индексируются байтами M8, а не кодовыми точками Unicode. Формат поддерживает моноширинные и пропорциональные глифы, а также независимый выбор сырого или RLE-представления для каждого глифа. Конвертер принимает читаемый UTF-8 на компьютере, но записывает в файл только представимые в M8 символы.

Этот документ описывает именно бинарный формат. Эталонные реализации находятся в `code/fmk_font.cpp`, `code/fmk_font.hpp` и `tools/fonts/fmk_font.cpp`.

Краткие ограничения

Параметр	Ограничение
Размер валидного FMK2	19..8192 байта; продукт может задать меньший предел
Коды глифов	Печатные байты M8; до 240 разных знаков
Диапазоны кодов	1..255
Символов в одном диапазоне	1..256
Число глифов	1..240 при действующих правилах M8
Максимальная ширина	1..16 пикселей
Высота	1..32 пикселя
Шаг по горизонтали	1..16 пикселей
Межстрочный интервал	0..15 пикселей

Все многобайтовые целые записываются в little-endian. Биты внутри битового потока записываются от старшего к младшему, то есть сначала маска `0x80`, затем `0x40` и так далее.

Общая структура файла

```
+----------------------+ 0
| заголовок, 16 байт   |
+----------------------+ 16
| диапазоны, 2 * R     |
+----------------------+
| битовые записи       |
| глифов без выравнивания
+----------------------+ size
```

`R` - число диапазонов из байта 10 заголовка. Первая запись глифа начинается сразу после последнего диапазона. Отдельного массива смещений нет: чтобы найти глиф с индексом `N`, декодер последовательно пропускает записи `0..N-1`.

Заголовок

Смещение	Размер	Поле	Описание
0	4	`magic`	ASCII `FMK2`
4	1	`flags`	Бит 0: моноширинный шрифт; биты 1..7 равны нулю
5	1	`max_width`	Ширина ячейки или максимальная ширина глифа, 1..16
6	1	`height`	Общая высота глифов, 1..32
7	1	`advance_gap`	Старшие 4 бита: `default_advance - 1`; младшие 4 бита: `line_gap`
8	2	`glyph_count`	Общее число глифов, little-endian
10	1	`range_count`	Число диапазонов, 1..255
11	1	`reserved`	Зарезервировано, обязательно 0
12	2	`file_size`	Полный размер файла, little-endian
14	2	`crc16`	CRC-16/CCITT-FALSE всего файла

Единственный определённый флаг:

```
FLAG_MONOSPACED = 0x01
```

Неизвестный установленный флаг делает файл недействительным. Для моноширинного шрифта ширина каждого глифа равна `max_width`, а шаг равен `default_advance`; индивидуальные метрики в потоке отсутствуют. Для пропорционального шрифта `max_width` задаёт верхнюю границу ширины, а метрики хранятся в каждой записи.

`file_size` должен в точности совпадать с фактической длиной массива. Это отсекает как усечённые данные, так и дописанный хвост.

CRC-16

Используется CRC-16/CCITT-FALSE:

- начальное значение `0xFFFF`;
- полином `0x1021`;
- обработка без отражения битов;
- без финального XOR;
- байты 14 и 15 при расчёте считаются нулевыми.

CRC покрывает заголовок, каталог диапазонов, записи глифов и последние биты заполнения. После формирования файла конвертер записывает размер, рассчитывает CRC с нулями в поле `crc16` и затем помещает результат в байты 14..15.

Псевдокод одного шага:

```
crc = 0xFFFF
for byte in file:
    if offset == 14 or offset == 15:
        byte = 0
    crc ^= byte << 8
    repeat 8 times:
        crc = (crc << 1) ^ 0x1021, if old bit 15 was 1
        crc = (crc << 1),          otherwise
```

Во всех операциях `crc` остаётся 16-битным.

Каталог диапазонов M8

После заголовка идут `range_count` двухбайтовых записей:

Смещение внутри записи	Размер	Поле	Описание
0	1	`first`	Первый байт M8 диапазона
1	1	`count_minus_1`	Число байтов минус 1

Фактическое число глифов диапазона равно `count_minus_1 + 1`. Диапазоны должны быть отсортированы по `first`, не должны пересекаться и не могут выходить за `0xFF`. Каждый байт обязан быть печатным M8: TAB, LF и CR не могут быть глифами. Сумма длин диапазонов должна в точности равняться `glyph_count`.

Диапазоны не содержат индексов. Индексы назначаются неявно и подряд: первый диапазон начинается с индекса 0, следующий - сразу после него. Например:

```
20 5E    ASCII 0x20..0x7E, 95 глифов
A8 00    Ё (M8 0xA8),       1 глиф
B8 00    ё (M8 0xB8),       1 глиф
C0 3F    А..я (M8 0xC0..0xFF), 64 глифа
```

Конвертер объединяет соседние байты M8, но разрывает диапазон после 256 символов, поскольку длина хранится в одном байте. Unicode используется только на входе host-конвертера и для обозначения символа человеку; ни кодовые точки, ни UTF-8-последовательности в каталоге FMK2 не хранятся.

Запись глифа

Записи следуют в порядке индексов и не выравниваются по границе байта. У всех глифов одна высота `height`. Пиксели идут построчно: сверху вниз, внутри строки слева направо.

Метрики пропорционального глифа

Если `FLAG_MONOSPACED` сброшен, запись начинается с 8 бит:

Биты	Значение
7..4	`width - 1`, фактическая ширина 1..16
3..0	`advance - 1`, фактический шаг 1..16

Ширина обязана быть не больше `max_width`. В моноширинном варианте этих 8 бит нет.

Сразу после метрик, либо в начале моноширинной записи, расположен бит режима bitmap:

- `0` - сырые пиксели;
- `1` - битовый RLE.

Сырые пиксели

После нулевого бита режима идут ровно `width * height` пикселей по одному биту. `0` означает фон, `1` - закрашенный пиксель. Строки на диске не дополняются до целого байта.

Стоимость записи bitmap без метрик:

```
1 + width * height бит
```

RLE

RLE-поток разворачивается ровно в `width * height` пикселей. Каждый токен начинается с бита вида.

Литеральный токен:

```
0 llll p...
```

- `llll` - 4-битное значение `length - 1`;
- длина литерала - 1..16 пикселей;
- после длины идут `length` исходных пикселей.

Токен серии:

```
1 lllll p
```

- `lllll` - 5-битное значение `length - 2`;
- длина серии - 2..33 пикселя;
- p - единственный пиксель, повторяемый `length` раз.

Токены не имеют собственного признака конца. Декодер прекращает чтение, когда получил требуемое число пикселей. Токен, который переполняет bitmap, является ошибкой.

Выбор сжатия

Конвертер решает задачу минимальной стоимости динамическим программированием отдельно для каждого глифа. Для каждой позиции рассматриваются:

- литералы длиной 1..16, стоимость `5 + length` бит;
- серии одинаковых битов длиной 2..33, стоимость 7 бит.

Режим `auto` выбирает RLE только если оптимальные RLE-токены строго короче сырых пикселей. Равная стоимость приводит к сырой записи. Общий бит режима есть в обоих вариантах и на решение не влияет.

Режимы конвертера:

Значение `--compression`	Поведение
`auto`	RLE только при реальном выигрыше, значение по умолчанию
`none`	Всегда сырые пиксели
`rle`	Всегда RLE, даже если файл станет больше

Таким образом, в одном FMK2-файле часть глифов может быть сырой, а часть - сжатой.

Завершение битового потока

После последнего пикселя последнего глифа может оставаться от 0 до 7 битов до границы байта. Все они обязаны быть нулевыми. Полный дополнительный байт заполнения запрещён: после разбора записей остаток должен быть меньше 8 бит.

Это правило вместе с `file_size` и CRC устраняет неоднозначный хвост и позволяет строго проверить весь файл.

Представление bitmap в RAM

Метод `Face::decode` выдаёт строки, выровненные до целого байта:

```
stride = (width + 7) / 8
required = stride * height
```

Внутри строки левый пиксель соответствует `0x80`. Неиспользуемые младшие биты последнего байта строки равны нулю. Максимальный буфер декодирования равен `2 * 32 = 64` байтам для глифа 16x32.

Важно различать это RAM-представление и непрерывный, не выровненный по строкам битовый поток в файле.

Строгая проверка файла прошивкой

`fmk::Face::open` принимает файл только после всех проверок:

1. Указатель не равен `NULL`, первичная проверка размера даёт 16..8192; после обязательного диапазона и bitmap минимальный валидный FMK2 занимает 19 байт.
2. Сигнатура равна `FMK2`, неизвестных флагов нет.
3. Геометрия, шаг, число глифов и число диапазонов допустимы.
4. Зарезервированный байт равен нулю.
5. Объявленный размер и CRC совпадают.
6. Диапазоны отсортированы, не пересекаются, состоят только из допустимых печатных байтов M8 и покрывают ровно все глифы.
7. Все метрики и bitmap-потоки разбираются без выхода за конец.
8. Последний неполный байт содержит только нулевое заполнение.

Смещения глифов не кэшируются: поиск по байту M8 сначала находит индекс в диапазонах, затем последовательно проходит поток до нужной записи. Это экономит RAM и подходит для небольшого лимита файла.

Конвертер FreeType

Сборка и запуск:

```
tools/fonts/build_fmk_font.sh INPUT OUTPUT [options]
```

Скрипт собирает внутренний `tools/fonts/fmk_font.cpp` через C++17 и `pkg-config freetype2`, если бинарник отсутствует или устарел. Готовая host-утилита кэшируется в `.build/tools/fmk_font`, а не в каталоге исходников.

Основные параметры:

Параметр	Описание
`--cell WxH`	Целевая ячейка, по умолчанию `5x8`; диапазон `1x1..16x32`
`--size N`	Размер FreeType в пикселях; по умолчанию `max(8, 4 * H)`
`--chars SPEC`	Набор символов
`--threshold N`	Явный порог покрытия 0..255; без параметра базовый порог 128 автоматически компенсируется при уменьшении растра
`--line-gap N`	Межстрочный интервал 0..15
`--proportional`	Индивидуальные ширина и шаг
`--compression MODE`	`auto`, `none` или `rle`
`--max-file-size N`	Целевой предел 16..8192; по умолчанию 1536, для UI F411 задаётся 8192

`--chars` принимает элементы через запятую; это запись набора символов для host-конвертера, а не кодировка FMK2:

- `ascii` - `U+0020..U+007E`;
- `latin1` - `U+0020..U+00FF`;
- `cyrillic` - Ё, А..я, ё;
- `U+XXXX` или `U+XXXX-U+YYYY`;
- произвольный UTF-8-текст.

Повторы удаляются, Unicode-символы преобразуются в M8, а глифы упорядочиваются по полученным байтам. Символы вне BMP отклоняются. Если рисунка нет во входном шрифте, символ пропускается с предупреждением; если рисунок есть, но символ невозможно представить в M8, конвертер выдаёт ошибку. Если не осталось ни одного глифа, конвертация завершается ошибкой.

Примеры:

```
tools/fonts/build_fmk_font.sh font.otf font.fmk \
  --cell 5x8 --chars ascii,cyrillic

tools/fonts/build_fmk_font.sh font.bdf font-3x5.fmk \
  --cell 3x5 --size 20 --chars U+0020-U+007E

tools/fonts/build_fmk_font.sh font.ttf DejaVu-12.FMK \
  --cell 10x12 --size 12 --line-gap 1 --proportional \
  --chars 'ascii,cyrillic,°×÷≤≥←→↑…π√' \
  --compression auto --max-file-size 8192
```

Растеризация

Конвертер сначала строит общий baseline-box по всем доступным запрошенным глифам. Это сохраняет взаимное вертикальное выравнивание. Затем покрытие каждого целевого пикселя вычисляется усреднением исходной маски по соответствующей прямоугольной области и сравнивается с `threshold`. Если `--threshold` не задан, базовое значение 128 делится на максимальный линейный коэффициент уменьшения, ограниченный стандартным коэффициентом supersampling 4. Это не дает тонким штрихам исчезнуть при стандартном supersampling; явно заданный `--threshold` применяется без такой коррекции.

В моноширинном режиме сохраняется вся ячейка. В пропорциональном режиме пустые крайние столбцы обрезаются; `advance` устанавливается в `width + 1`, но не более 16 пикселей. Полностью пустой глиф получает ненулевую ширину `max(1, cell_width / 2)`.

По умолчанию `line_gap` равен 1 для высоты до 5, 2 для высоты 6..8 и 0 для большей высоты.

Использование в MK61s mini

Во внутреннем хранилище FMK2 имеет тип `FONT` с двухбайтовой сигнатурой `f1`; на виртуальном USB-диске используется расширение `.fmk`. Строчная `f` отличает семейство шрифтов от FOCAL `F1`. F401 сохраняет предел файла 1536 байт; F411 принимает до 8192 байт. Предел конвертера остаётся 1536 по умолчанию и расширяется только явным `--max-file-size 8192`.

FMK2 является дисковым форматом, но не внутренним форматом рендерера. `SETUP.APP` один раз проверяет заголовок, диапазоны, CRC и RLE, разворачивает глифы в индексированный RAM-образ `PFK2`, а резидент повторно проверяет этот образ и только затем атомарно копирует его в общую арену `BULK`. `PFK2` на диск не записывается и не является публичным форматом совместимости. Поэтому FMK-декодера и RLE-парсера в резидентной прошивке нет, а рисование глифа не сканирует предыдущие записи и не распаковывает bitmap заново. Поиск в `PFK2` идёт по байту M8; рендерер преобразует кодовую точку экранной сетки в M8 на границе lookup, не хранит Unicode в файле шрифта.

Ограничение RAM относится к подготовленному образу: он также должен уместиться в `BULK` (1536 байт на F401 и 8192 байта на F411). Хорошо сжатый FMK может развернуться в более крупный PFK2 и быть корректным как файл, но не подходить конкретной плате; в этом случае старый активный шрифт остаётся нетронутым.

FMK2 уже имеет собственное сжатие. Поэтому объектное хранилище не применяет к типу `FONT` внешний LZSS и считает установленный флаг внешнего сжатия ошибкой. Подробности внешнего слоя приведены в `MK61s-mini-Storage.md`.

На графическом UC1609 произвольный файл по-прежнему можно просмотреть и командой Run установить как шрифт обычной фиксированной текстовой сетки. Для интерфейса UC1609 на F401 и F411 предусмотрен сохраняемый каталог: каждый непосредственный файл /Fonts/*.FMK является самостоятельным пунктом, а его именем в меню служит имя файла без расширения. Размер 12/14/16 читается из самого заголовка; несколько размеров одной гарнитуры оформляются несколькими файлами. Здесь advance действительно задаёт переменный шаг между знаками. Обычный отсутствующий символ заменяется глифом ?, а внутренние токены интерфейса могут использовать встроенный 5x8.

UI-файл дополнительно обязан иметь высоту 12, 14 или 16, line_gap не более 4, не менее трёх строк на матрице 192x64 и содержать пробел и ?. Выбор переживает перезапуск и сохраняется по стабильному ключу имени, а не по номеру узла или позиции каталога. Повреждённый, удалённый, переименованный или не подходящий по метрикам файл не становится видимым рендереру: интерфейс возвращается на встроенный Pixel того же размера. При включении USB-диска PFK2 временно освобождает общую арену, после выхода исходный FMK заново находится по имени и компилируется через SETUP.APP. Старые корневые UI12/14/16.FMK остаются только миграционным резервом прежних настроек.

Подготовленные образы обычного внешнего шрифта и внешнего UI-шрифта делят одну арену, поэтому одновременно активна только одна роль. Явный Run обычного шрифта переводит UI на Pixel; выбор файла из /Fonts заменяет обычный внешний шрифт. Индикатор калькулятора на UC1609 (как F401, так и F411) имеет собственный встроенный растр и не меняется.

На LCD1602 установка внешнего шрифта не поддерживается. Режим просмотра выбирает до восьми глифов, масштабирует их nearest-neighbor до 5x8 и временно занимает все восемь слотов CGRAM. Пропорциональный файл на LCD1602 отклоняется режимом просмотра.

Минимальный алгоритм декодирования

```
validate_header_ranges_crc()
bitpos = (16 + 2 * range_count) * 8

for each glyph:
    if proportional:
        width   = read_bits(4) + 1
        advance = read_bits(4) + 1
    else:
        width   = max_width
        advance = default_advance

    mode = read_bits(1)
    if mode == 0:
        read width * height pixels
    else:
        while produced < width * height:
            kind = read_bits(1)
            if kind == 0:
                count = read_bits(4) + 1
                read count literal pixels
            else:
                count = read_bits(5) + 2
                pixel = read_bits(1)
                emit pixel count times

require fewer than 8 remaining bits
require all remaining bits == 0
```

Декодер обязан проверять каждое сложение и длину до чтения данных. CRC не заменяет структурную проверку: корректная контрольная сумма у логически недопустимого потока не должна приводить к выходу за границы буфера.